cawlign

0.0.1

Generated by Doxygen 1.15.0

1 Hierarchical Index 1

1.1 Class Hierarchy . . . 1

2 Class Index 3

2.1 Class List . . . 3

3 File Index 5

3.1 File List . . . 5

4 Class Documentation 7

4.1 argparse::args_t Class Reference . . . 7

4.1.1 Constructor & Destructor Documentation . . . 7

4.1.1.1 args_t() . . . 7

4.1.1.2 ~args_t() . . . 8

4.2 CawalignCodonScores Class Reference . . . 8

4.2.1 Constructor & Destructor Documentation . . . 9

4.2.1.1 CawalignCodonScores() . . . 9

4.2.2 Member Function Documentation . . . 10

4.2.2.1 nucleotide_diff() . . . 10

4.3 CawalignSimpleScores Class Reference . . . 10

4.3.1 Constructor & Destructor Documentation . . . 11

4.3.1.1 CawalignSimpleScores() [1/2] . . . 11

4.3.1.2 CawalignSimpleScores() [2/2] . . . 11

4.3.2 Member Function Documentation . . . 11

4.3.2.1 _init_alphabet() . . . 11

4.4 ConfigParser Class Reference . . . 12

4.4.1 Constructor & Destructor Documentation . . . 12

4.4.1.1 ConfigParser() . . . 12

4.4.2 Member Function Documentation . . . 13

4.4.2.1 aConfig() . . . 13

4.4.2.2 aConfigVec() . . . 13

4.5 KmerHit Struct Reference . . . 14

4.6 RefKmerEntry Struct Reference . . . 14

4.7 StringBuffer Class Reference . . . 14

4.7.1 Detailed Description . . . 15

4.7.2 Constructor & Destructor Documentation . . . 15

4.7.2.1 StringBuffer() . . . 15

4.7.2.2 ~StringBuffer() . . . 16

4.7.3 Member Function Documentation . . . 16

4.7.3.1 appendBuffer() . . . 16

4.7.3.2 appendChar() . . . 16

4.7.3.3 flip() . . . 17

4.7.3.4 getChar() . . . 17

4.7.3.5 getString() . . . 17
4.7.3.6 length() . . . 17
4.7.3.7 reset_length() . . . 18
4.7.3.8 resetString() . . . 18
4.7.3.9 setChar() . . . 18
4.7.3.10 swap() . . . 19
4.8 Vector Class Reference . . . 19
4.8.1 Detailed Description . . . 20
4.8.2 Constructor & Destructor Documentation . . . 20
4.8.2.1 Vector() . . . 20
4.8.2.2 ∼Vector() . . . 20
4.8.3 Member Function Documentation . . . 21
4.8.3.1 appendValue() . . . 21
4.8.3.2 appendVector() . . . 21
4.8.3.3 extractMin() . . . 21
4.8.3.4 length() . . . 22
4.8.3.5 remove() . . . 22
4.8.3.6 resetVector() . . . 23
4.8.3.7 sort() . . . 23
4.8.3.8 storeValue() . . . 23
4.8.3.9 storeVector() . . . 24
4.8.3.10 swap() . . . 24
4.8.3.11 value() . . . 24
4.9 VectorFP Class Reference . . . 25
4.9.1 Detailed Description . . . 26
4.9.2 Constructor & Destructor Documentation . . . 26
4.9.2.1 VectorFP() . . . 26
4.9.2.2 ∼VectorFP() . . . 26
4.9.3 Member Function Documentation . . . 26
4.9.3.1 appendValue() . . . 26
4.9.3.2 appendValues() . . . 27
4.9.3.3 length() . . . 27
4.9.3.4 rvalues() . . . 27
4.9.3.5 storeValue() . . . 28
4.9.3.6 value() . . . 28
4.9.3.7 values() . . . 28

5 File Documentation **29**

5.1 alignment.h . . . 29
5.2 argparse.hpp . . . 30
5.3 configparser.hpp . . . 32
5.4 scoring.hpp . . . 33

5.5 stringBuffer.h 34
5.6 tn93_shared.h 35

Index **37**

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

argparse::args_t 7
CawalignSimpleScores 10
    CawalignCodonScores 8
ConfigParser 12
KmerHit 14
RefKmerEntry 14
StringBuffer 14
Vector 19
VectorFP 25

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

argparse::args_t 7
CawalignCodonScores 8
CawalignSimpleScores 10
ConfigParser 12
KmerHit 14
RefKmerEntry 14
StringBuffer
A simple string buffer class that allows for dynamic resizing 14
Vector
A simple vector class for long integers that allows for dynamic resizing 19
VectorFP
A simple vector class for floating point numbers that allows for dynamic resizing 25

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

src/alignment.h 29
src/argparse.hpp 30
src/configparser.hpp 32
src/scoring.hpp 33
src/stringBuffer.h 34
src/tn93_shared.h 35

Chapter 4

Class Documentation

4.1 argparse::args_t Class Reference

Public Member Functions

- args_t (int, const char **)
- ~args_t ()

Public Attributes

- FILE * **output**
- FILE * **reference**
- FILE * **input**
- ConfigParser * **scores**
- data_t **data_type**
- local_t **local_option**
- space_t **space_type**
- out_format_t **out_format**
- rc_t **reverse_complement**
- ConfigParser * **genetic_code**
- bool **quiet**
- bool **affine**
- bool **include_reference**
- StringBuffer * **memory_ref**

4.1.1 Constructor & Destructor Documentation

4.1.1.1 args_t()

```
argparse::args_t::args_t (
            int ,
            const char ** )
```

Constructor for args_t, which parses command-line arguments and sets up configuration options.

This constructor processes the command-line arguments, setting up the program's input, output, reference files, and various other configuration options such as data type, space type, and output format. If necessary, default values are assigned to some options.

Parameters

argc	The number of command-line arguments.
argv	The array of command-line arguments.

4.1.1.2 ~args_t()

```
argparse::args_t::~args_t ()
```

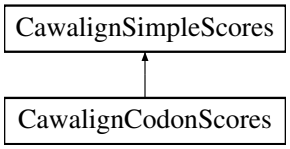
Destructor for args_t, responsible for cleaning up any resources used (files or memory). It closes the input/output/reference files and deletes the scores object, if applicable.

The documentation for this class was generated from the following files:

- src/argparse.hpp
- src/argparse.cpp

4.2 CawalignCodonScores Class Reference

Inheritance diagram for CawalignCodonScores:



Public Member Functions

- CawalignCodonScores (ConfigParser *, ConfigParser *)

Public Member Functions inherited from CawalignSimpleScores

- CawalignSimpleScores (const char *_alphabet, const cawlign_fp *_scoring_matrix, const cawlign_fp _↩ open_gap_reference, const cawlign_fp _open_gap_query, const cawlign_fp _extend_gap_reference, const cawlign_fp _extend_gap_query)
- CawalignSimpleScores (ConfigParser *)
- void _init_alphabet (long not_found=-1)

Static Public Member Functions

- static int nucleotide_diff (long, long)

Public Attributes

- Vector **translation_table**
- Vector **resolutions**
- VectorFP **s3x1**
- VectorFP **s3x2**
- VectorFP **s3x4**
- VectorFP **s3x5**
- cawlign_fp **frameshift_cost**
- cawlign_fp **synonymous_penalty**
- StringBuffer **amino_acids**
- int **stop_codon_index**
- int **mismatch_index**

Public Attributes inherited from CawalignSimpleScores

- StringBuffer **alphabet**
- unsigned int **D**
- long **char_map** [255]
- VectorFP **scoring_matrix**
- cawlign_fp **open_gap_reference**
- cawlign_fp **open_gap_query**
- cawlign_fp **extend_gap_query**
- cawlign_fp **extend_gap_reference**
- char **gap_char**

4.2.1 Constructor & Destructor Documentation

4.2.1.1 CawalignCodonScores()

```
CawalignCodonScores::CawalignCodonScores (
            ConfigParser * code_settings,
            ConfigParser * score_settings)
```

Constructs a `CawalignCodonScores` object using configuration score_settings.

This constructor initializes the codon scoring system using values from a `ConfigParser`. It sets up the codon translation table, stop codon index, mismatch index, and scoring matrices for codon alignments. Throws errors if the amino acid alphabet is incomplete or the translation table is invalid.

Parameters

code_settings	A pointer to a `ConfigParser` object containing genetic code settings.
score_settings	A pointer to a `ConfigParser` object containing configuration score_settings.

4.2.2 Member Function Documentation

4.2.2.1 nucleotide_diff()

```
int CawalignCodonScores::nucleotide_diff (
            long c1,
            long c2)  [static]
```

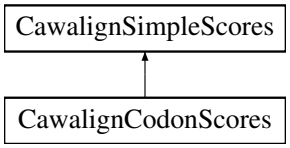
Calculates the number of nucleotide differences between two codons.

The documentation for this class was generated from the following files:

- src/scoring.hpp
- src/scoring.cpp

4.3 CawalignSimpleScores Class Reference

Inheritance diagram for CawalignSimpleScores:



Public Member Functions

- CawalignSimpleScores (const char *_alphabet, const cawlign_fp *_scoring_matrix, const cawlign_fp _open_gap_reference, const cawlign_fp _open_gap_query, const cawlign_fp _extend_gap_reference, const cawlign_fp _extend_gap_query)
- CawalignSimpleScores (ConfigParser *)
- void _init_alphabet (long not_found=-1)

Public Attributes

- StringBuffer **alphabet**
- unsigned int **D**
- long **char_map** [255]
- VectorFP **scoring_matrix**
- cawlign_fp **open_gap_reference**
- cawlign_fp **open_gap_query**
- cawlign_fp **extend_gap_query**
- cawlign_fp **extend_gap_reference**
- char **gap_char**

4.3.1 Constructor & Destructor Documentation

4.3.1.1 CawalignSimpleScores() [1/2]

```
CawalignSimpleScores::CawalignSimpleScores (
            const char * _alphabet,
            const cawlign_fp * _scoring_matrix,
            const cawlign_fp _open_gap_reference,
            const cawlign_fp _open_gap_query,
            const cawlign_fp _extend_gap_reference,
            const cawlign_fp _extend_gap_query)
```

Constructs a CawalignSimpleScores object with a custom alphabet and scoring matrix.

This constructor initializes the scoring system with a user-specified alphabet and scoring matrix, as well as gap penalties for both reference and query sequences. Throws an error if the alphabet is empty.

Parameters

_alphabet	The alphabet for scoring (e.g., nucleotides or amino acids).
_scoring_matrix	A pointer to the scoring matrix values.
_open_gap_reference	Gap opening penalty for reference sequences.
_open_gap_query	Gap opening penalty for query sequences.
_extend_gap_reference	Gap extension penalty for reference sequences.
_extend_gap_query	Gap extension penalty for query sequences.

4.3.1.2 CawalignSimpleScores() [2/2]

```
CawalignSimpleScores::CawalignSimpleScores (
            ConfigParser * score_settings)
```

Constructs a CawalignSimpleScores object using configuration score_settings.

This constructor reads configuration values from a ConfigParser to initialize the alphabet, scoring matrix, and gap penalties. Throws errors if the alphabet is missing or the scoring matrix dimensions are incorrect.

Parameters

score_settings	A pointer to a ConfigParser object containing configuration score settings.

4.3.2 Member Function Documentation

4.3.2.1 _init_alphabet()

```
void CawalignSimpleScores::_init_alphabet (
            long not_found = -1)
```

Initializes the character map for the scoring matrix.

This function populates the `char_map` array, mapping each character in the alphabet to its index in the scoring matrix. Characters not in the alphabet are assigned a value of `not_found`.

Parameters

not_found	The value to assign for characters not found in the alphabet.

The documentation for this class was generated from the following files:

- src/scoring.hpp
- src/scoring.cpp

4.4 ConfigParser Class Reference

Public Member Functions

- ConfigParser (std::ifstream &configFile)
- bool **hasConfig** (std::string section, std::string name) const
- template<typename T>
  T **aConfig** (std::string section, std::string name, size_t pos=0)
- template<typename T>
  std::vector< T > **aConfigVec** (std::string section, std::string name)
- template<> bool aConfig (std::string section, std::string configName, size_t pos)
- template<> std::vector< bool > aConfigVec (std::string section, std::string configName)
- template<> bool **aConfig** (std::string section, std::string name, size_t pos)
- template<> std::vector< bool > **aConfigVec** (std::string section, std::string name)

4.4.1 Constructor & Destructor Documentation

4.4.1.1 ConfigParser()

```
ConfigParser::ConfigParser (
            std::ifstream & configFile)
```

Constructor for `ConfigParser`.

This function reads and parses a configuration file. It processes lines to strip whitespace, ignores comments, and splits key-value pairs using the = symbol. If the key is within a section (indicated by `[...]`), the key is prefixed with the section name. Multiple values for a key are stored in a vector. Parsed configurations are stored in a map with section and key names combined as the key.

Parameters

configFile	A reference to an `ifstream` representing the open configuration file.

Exceptions

std::runtime_error	if a parsing error occurs in the configuration file.

4.4.2 Member Function Documentation

4.4.2.1 aConfig()

```
template<>
bool ConfigParser::aConfig (
            std::string section,
            std::string configName,
            size_t pos)
```

Template specialization for retrieving boolean configuration values.

This function retrieves a specific boolean configuration value from a section and configuration name. The value is parsed as either `true` (for "true", "TRUE", or "1") or `false` (for "false", "FALSE", or "0"). If the value does not match any of these, the function defaults to `false`.

Parameters

section	The section name in the configuration file.
configName	The key within the section.
pos	The position of the value in the vector (if there are multiple values).

Returns

`true` if the configuration value is recognized as true, otherwise `false`.

4.4.2.2 aConfigVec()

```
template<>
std::vector< bool > ConfigParser::aConfigVec (
            std::string section,
            std::string configName)
```

Template specialization for retrieving a vector of boolean configuration values.

This function retrieves a vector of boolean values associated with a configuration key in a section. Each value is parsed as either `true` (for "true", "TRUE", or "1") or `false` (for "false", "FALSE", or "0"). If a value does not match any of these, it defaults to `false`.

Parameters

section	The section name in the configuration file.
configName	The key within the section.

Returns

A vector of boolean values parsed from the configuration.

The documentation for this class was generated from the following files:

- src/configparser.hpp
- src/configparser.cpp

4.5 KmerHit Struct Reference

Public Attributes

- int32_t **r**
- int32_t **q**
- int32_t **d**

The documentation for this struct was generated from the following file:

- src/alignment.cpp

4.6 RefKmerEntry Struct Reference

Public Attributes

- int32_t **pos1**
- int32_t **pos2**

The documentation for this struct was generated from the following file:

- src/alignment.cpp

4.7 StringBuffer Class Reference

A simple string buffer class that allows for dynamic resizing.

`#include <stringBuffer.h>`

Public Member Functions

- StringBuffer (void)

  Construct a new String Buffer object.
- ~StringBuffer (void)

  Destroy the String Buffer object.
- char * getString (void) const

  Get the underlying C string.
- void appendChar (const char)

  Append a character to the string buffer.
- void appendBuffer (const char *, const long=-1)

  Append a C string to the string buffer.
- void resetString (void)

  Reset the string buffer to an empty state.
- void swap (StringBuffer &)

  Swap the contents of this string buffer with another.
- unsigned long length (void) const

  Get the length of the string buffer.
- void reset_length (unsigned long newL)

  Reset the length of the string buffer.
- char setChar (const long i, const char c)

  Set a character at a specific index.
- char getChar (const long i) const

  Get a character at a specific index.
- void flip (void)

  Flip the string buffer in place.
- void **detach** (void)

  Detach the underlying C string from the string buffer.

Static Public Attributes

- static long **sbDefaultLength** = 16
- static long **sbDefaultBoost** = 16

4.7.1 Detailed Description

A simple string buffer class that allows for dynamic resizing.

4.7.2 Constructor & Destructor Documentation

4.7.2.1 StringBuffer()

```
StringBuffer::StringBuffer (
            void )
```

Construct a new String Buffer object.

Initializes an empty `StringBuffer` with a default initial capacity. This buffer dynamically grows as new characters are appended.

4.7.2.2 ~StringBuffer()

```
StringBuffer::~StringBuffer (
            void )
```

Destroy the String Buffer object.

Frees the memory allocated for the string buffer.

4.7.3 Member Function Documentation

4.7.3.1 appendBuffer()

```
void StringBuffer::appendBuffer (
            const char * buffer,
            const long length = -1)
```

Append a C string to the string buffer.

Parameters

buffer	The C string to append.
length	The length of the C string to append. If -1, the entire string is appended.

Appends a string or a buffer of specified length to the `StringBuffer`.

Parameters

buffer	The string or character buffer to append.
length	The length of the buffer, if known. If not, the length is inferred using `strlen`.

4.7.3.2 appendChar()

```
void StringBuffer::appendChar (
            const char c)
```

Append a character to the string buffer.

Parameters

c	The character to append.

Appends a single character to the end of the buffer, growing the buffer if needed.

Parameters

c	The character to append.

4.7.3.3 flip()

```
void StringBuffer::flip (
            void )
```

Flip the string buffer in place.

Reverses the content of the `StringBuffer`.

4.7.3.4 getChar()

```
char StringBuffer::getChar (
            const long i) const  [inline]
```

Get a character at a specific index.

Parameters

i	The index to get the character from.

Returns

char The character at the index.

4.7.3.5 getString()

```
char * StringBuffer::getString (
            void ) const  [inline]
```

Get the underlying C string.

Returns

char∗ A pointer to the underlying C string.

4.7.3.6 length()

```
unsigned long StringBuffer::length (
            void ) const  [inline]
```

Get the length of the string buffer.

Returns

unsigned long The length of the string buffer.

4.7.3.7 reset_length()

```
void StringBuffer::reset_length (
            unsigned long newL)  [inline]
```

Reset the length of the string buffer.

Parameters

newL	The new length of the string buffer.

4.7.3.8 resetString()

```
void StringBuffer::resetString (
            void )
```

Reset the string buffer to an empty state.

Resets the `StringBuffer` to an empty state.

Clears the buffer content by resetting its length, but keeps the allocated memory.

4.7.3.9 setChar()

```
char StringBuffer::setChar (
            const long i,
            const char c)  [inline]
```

Set a character at a specific index.

Parameters

i	The index to set the character at.
c	The character to set.

Returns

char The character that was previously at the index.

4.7.3.10 swap()

```
void StringBuffer::swap (
            StringBuffer & src)
```

Swap the contents of this string buffer with another.

Parameters

other	The other string buffer to swap with.

Swaps the contents of this `StringBuffer` with another `StringBuffer`.

This function exchanges the data, length, and capacity of two `StringBuffer` objects.

Parameters

src	The `StringBuffer` object to swap with.

The documentation for this class was generated from the following files:

- src/stringBuffer.h
- src/stringBuffer.cc

4.8 Vector Class Reference

A simple vector class for long integers that allows for dynamic resizing.

```
#include <stringBuffer.h>
```

Public Member Functions

- Vector (void)
  Construct a new Vector object.
- ~Vector (void)
  Destroy the Vector object.
- void appendValue (const long)
  Append a value to the vector.
- void appendVector (const Vector &)
  Append another vector to this vector.
- long extractMin (VectorFP &)
  Extract the minimum value from a VectorFP object.
- void resetVector (void)
  Reset the vector to an empty state.
- void remove (const unsigned long)
  Remove a value at a specific index.
- void storeValue (const long, const unsigned long)
  Store a value at a specific index.

- void storeVector (const Vector &, const unsigned long)

  Store another vector at a specific index.
- void sort (void)

  Sort the vector in place.
- void swap (Vector &)

  Swap the contents of this vector with another.
- long value (const long idx) const

  Get a value at a specific index.
- unsigned long length (void) const

  Get the length of the vector.
- const long * **rvalues** (void) const

Static Public Attributes

- static long **vDefaultLength** = 16
- static long **vDefaultBoost** = 16

4.8.1 Detailed Description

A simple vector class for long integers that allows for dynamic resizing.

4.8.2 Constructor & Destructor Documentation

4.8.2.1 Vector()

```
Vector::Vector (
            void )
```

Construct a new Vector object.

Initializes an empty vector of `long` values with a default initial capacity.

4.8.2.2 ~Vector()

```
Vector::~Vector (
            void )
```

Destroy the Vector object.

Frees the memory allocated for the vector data.

4.8.3 Member Function Documentation

4.8.3.1 appendValue()

```
void Vector::appendValue (
            const long l)
```

Append a value to the vector.

Parameters

value	The value to append.

Appends a value to the Vector.

Adds a long value to the end of the vector, growing the vector if needed.

Parameters

l	The long value to append.

4.8.3.2 appendVector()

```
void Vector::appendVector (
            const Vector & v)
```

Append another vector to this vector.

Parameters

other	The other vector to append.

Appends the contents of another Vector to this Vector.

Parameters

v	The source Vector whose contents are to be appended.

4.8.3.3 extractMin()

```
long Vector::extractMin (
            VectorFP & values)
```

Extract the minimum value from a VectorFP object.

Parameters

other	The VectorFP object to extract the minimum value from.

Returns

long The minimum value.

Extracts the minimum value from the vector based on a `VectorFP` of floating-point values.

Parameters

values	The `VectorFP` of floating-point values to compare.

Returns

The index of the minimum value, or `-1` if the vector is empty.

4.8.3.4 length()

```
unsigned long Vector::length (
            void ) const  [inline]
```

Get the length of the vector.

Returns

unsigned long The length of the vector.

4.8.3.5 remove()

```
void Vector::remove (
            const unsigned long l)
```

Remove a value at a specific index.

Parameters

index	The index to remove the value at.

Removes the element at the given index and shifts the remaining elements to fill the gap.

Parameters

l	The index of the element to remove.

4.8.3.6 resetVector()

```
void Vector::resetVector (
            void )
```

Reset the vector to an empty state.

Resets the `Vector` to an empty state.

Clears the vector by resetting its length, but keeps the allocated memory.

4.8.3.7 sort()

```
void Vector::sort (
            void )
```

Sort the vector in place.

Sorts the `Vector` in ascending order.

4.8.3.8 storeValue()

```
void Vector::storeValue (
            const long v,
            const unsigned long l)
```

Store a value at a specific index.

Parameters

value	The value to store.
index	The index to store the value at.

Stores a value at a specific index in the `Vector`.

If the index is beyond the current capacity, the vector grows to accommodate the value.

Parameters

v	The `long` value to store.
l	The index at which to store the value.

4.8.3.9 storeVector()

```
void Vector::storeVector (
            const Vector & v,
            const unsigned long l)
```

Store another vector at a specific index.

Parameters

other	The other vector to store.
index	The index to store the vector at.

Stores a pointer to a `Vector` at a specified index, growing the vector if necessary.

Parameters

v	The `Vector` object to store.
l	The index at which to store the vector.

4.8.3.10 swap()

```
void Vector::swap (
            Vector & src)
```

Swap the contents of this vector with another.

Parameters

other	The other vector to swap with.

Swaps the contents of this `Vector` with another `Vector`.

Exchanges the data, length, and capacity of two `Vector` objects.

Parameters

src	The `Vector` object to swap with.

4.8.3.11 value()

```
long Vector::value (
            const long idx) const
```

Get a value at a specific index.

Parameters

idx	The index to get the value from.

Returns

long The value at the index.

Retrieves a value from the `Vector` at the specified index.

Parameters

idx	The index from which to retrieve the value.

Returns

The value stored at the specified index.

The documentation for this class was generated from the following files:

- src/stringBuffer.h
- src/stringBuffer.cc

4.9 VectorFP Class Reference

A simple vector class for floating point numbers that allows for dynamic resizing.

`#include <stringBuffer.h>`

Public Member Functions

- VectorFP (void)
  Construct a new VectorFP object.
- ~VectorFP (void)
  Destroy the VectorFP object.
- void appendValue (const cawlign_fp)
  Append a value to the vector.
- void appendValues (const cawlign_fp *, long)
  Append multiple values to the vector.
- void storeValue (const cawlign_fp, const unsigned long)
  Store a value at a specific index.
- cawlign_fp value (const long idx)
  Get a value at a specific index.
- unsigned long length (void) const
  Get the length of the vector.
- const cawlign_fp * values (void)
  Get the underlying C array.
- cawlign_fp * rvalues (void)
  Get the underlying C array.

Static Public Attributes

- static long **vDefaultLength** = 16
- static long **vDefaultBoost** = 16

4.9.1 Detailed Description

A simple vector class for floating point numbers that allows for dynamic resizing.

4.9.2 Constructor & Destructor Documentation

4.9.2.1 VectorFP()

```
VectorFP::VectorFP (
            void )
```

Construct a new VectorFP object.

Initializes an empty vector of floating-point values with a default initial capacity.

4.9.2.2 ~VectorFP()

```
VectorFP::~VectorFP (
            void )
```

Destroy the VectorFP object.

Frees the memory allocated for the floating-point vector data.

4.9.3 Member Function Documentation

4.9.3.1 appendValue()

```
void VectorFP::appendValue (
            const cawlign_fp l)
```

Append a value to the vector.

Parameters

value	The value to append.

Appends a floating-point value to the `VectorFP`.

Adds a floating-point value to the end of the vector, growing the vector if needed.

Parameters

l	The floating-point value to append.

4.9.3.2 appendValues()

```
void VectorFP::appendValues (
            const cawlign_fp * l,
            long N)
```

Append multiple values to the vector.

Parameters

values	The values to append.
count	The number of values to append.

Appends multiple floating-point values to the VectorFP.

Parameters

l	The array of floating-point values to append.
N	The number of values to append.

4.9.3.3 length()

```
unsigned long VectorFP::length (
            void ) const  [inline]
```

Get the length of the vector.

Returns

unsigned long The length of the vector.

4.9.3.4 rvalues()

```
cawlign_fp * VectorFP::rvalues (
            void )  [inline]
```

Get the underlying C array.

Returns

cawlign_fp* A pointer to the underlying C array.

4.9.3.5 storeValue()

```
void VectorFP::storeValue (
            const cawlign_fp v,
            const unsigned long l)
```

Store a value at a specific index.

Parameters

value	The value to store.
index	The index to store the value at.

Stores a floating-point value at a specific index in the VectorFP.

If the index is beyond the current capacity, the vector grows to accommodate the value.

Parameters

v	The floating-point value to store.
l	The index at which to store the value.

4.9.3.6 value()

```
cawlign_fp VectorFP::value (
            const long idx)  [inline]
```

Get a value at a specific index.

Parameters

idx	The index to get the value from.

Returns

cawlign_fp The value at the index.

4.9.3.7 values()

```
const cawlign_fp * VectorFP::values (
            void )  [inline]
```

Get the underlying C array.

Returns

const cawlign_fp* A pointer to the underlying C array.

The documentation for this class was generated from the following files:

- src/stringBuffer.h
- src/stringBuffer.cc

Chapter 5

File Documentation

5.1 alignment.h

```
00001 /*
00002
00003  HyPhy - Hypothesis Testing Using Phylogenies.
00004
00005  Copyright (C) 1997-now
00006  Core Developers:
00007  Sergei L Kosakovsky Pond (spond@ucsd.edu)
00008  Art FY Poon    (apoon42@uwo.ca)
00009  Steven Weaver (sweaver@ucsd.edu)
00010
00011  Module Developers:
00012  Lance Hepler (nlhepler@gmail.com)
00013  Martin Smith (martin.audacis@gmail.com)
00014
00015  Significant contributions from:
00016  Spencer V Muse (muse@stat.ncsu.edu)
00017  Simon DW Frost (sdf22@cam.ac.uk)
00018
00019  Permission is hereby granted, free of charge, to any person obtaining a
00020  copy of this software and associated documentation files (the
00021  "Software"), to deal in the Software without restriction, including
00022  without limitation the rights to use, copy, modify, merge, publish,
00023  distribute, sublicense, and/or sell copies of the Software, and to
00024  permit persons to whom the Software is furnished to do so, subject to
00025  the following conditions:
00026
00027  The above copyright notice and this permission notice shall be included
00028  in all copies or substantial portions of the Software.
00029
00030  THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
00031  OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
00032  MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
00033  IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
00034  CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
00035  TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
00036  SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00037
00038  */
00039
00040 #ifndef __ALIGNMENT_HEADER_FILE__
00041
00042 #define __ALIGNMENT_HEADER_FILE__
00043
00044 typedef   float     cawlign_fp;
00045
00046 cawlign_fp AlignStrings( char const * r_str
00047                     , char const * q_str
00048                     , const long _r_len
00049                     , const long _q_len
00050                     , char * & r_res
00051                     , char * & q_res
00052                     , long * char_map
00053                     , const cawlign_fp * cost_matrix
00054                     , const long cost_stride
00055                     , const char gap
00056                     , cawlign_fp open_insertion
00057                     , cawlign_fp extend_insertion
```

```
00058                     , cawlign_fp open_deletion
00059                     , cawlign_fp extend_deletion
00060                     , cawlign_fp miscall_cost
00061                     , const bool do_local
00062                     , const bool do_affine
00063                     , const bool do_codon
00064                     , const long char_count
00065                     , const cawlign_fp * codon3x5
00066                     , const cawlign_fp * codon3x4
00067                     , const cawlign_fp * codon3x2
00068                     , const cawlign_fp * codon3x1
00069                     , const bool do_true_local = false
00070                     , const bool report_ref_insertions = true
00071                     , cawlign_fp* score_matrix_cache = nullptr
00072                     , cawlign_fp* insertion_matrix_cache = nullptr
00073                     , cawlign_fp* deletion_matrix_cache = nullptr
00074                     , const long* resolution_map = nullptr
00075
00076                     );
00077
00078 cawlign_fp LinearSpaceAlign(    const char * s1           // first string
00079                             , const char * s2           // second string
00080                             , const long s1L
00081                             , const long s2L
00082                             , long*  cmap     // char -> position in scoring matrix mapper
00083                             , const cawlign_fp * ccost        // NxN matrix of edit distances on
      characters
00084                             , const long costD
00085                             , cawlign_fp gopen       // the cost of opening a gap in sequence 1
00086                             , cawlign_fp gextend     // the cost of extending a gap in sequence 1
      (ignored unless doAffine == true)
00087                             , cawlign_fp gopen2      // the cost of opening a gap in sequence 2
00088                             , cawlign_fp gextend2    // the cost of opening a gap in sequence 2
      (ignored unless doAffine == true)
00089                             , bool doLocal           // ignore prefix and suffix gaps
00090                             , bool doAffine          // use affine gap penalties
00091                             , long * ops      // edit operations for the optimal alignment
00092                             , cawlign_fp scoreCheck  // check the score of the alignment
00093                             , long from1
00094                             , long to1
00095                             , long from2
00096                             , long to2
00097                             , cawlign_fp ** buffer      // matrix storage,
00098                             , char parentGapLink
00099                             , char * ha
00100                             );
00101
00102 #endif
```

5.2 argparse.hpp

```
00001
00002 #ifndef ARGPARSE_H
00003 #define ARGPARSE_H
00004
00005 #include <stdio.h>
00006 #include <string>
00007 #include <configparser.hpp>
00008 // argument defaults
00009
00010 #define PROGNAME                "cawlign"
00011 #define DEFAULT_DATA_TYPE       nucleotide
00012 #define DEFAULT_GENETIC_CODE    "universal"
00013 #define DEFAULT_REFERENCE       "HXB2_pol"
00014 #define DEFAULT_SCORING         "Nucleotide-BLAST"
00015 #define DEFAULT_SPACE           quadratic
00016 #define DEFAULT_LOCAL_TYPE      trim
00017 #define DEFAULT_OUTPUT_FORMAT   refmap
00018 #define DEFAULT_RC_TYPE         none
00019
00020 #include "stringBuffer.h"
00021
00022 #ifndef VERSION_NUMBER
00023     #define VERSION_NUMBER           "0.2.0"
00024 #endif
00025
00026 #ifndef LIBRARY_PATH
00027     #define LIBRARY_PATH           "/usr/local/share/cawlign/"
00028 #endif
00029
00030 #define SCORES_SUBPATH        "scoring"
00031 #define REF_SUBPATH           "references"
00032 #define GENETIC_CODES_SUBPATH "genetic_codes"
```

```
00033
00034 namespace argparse
00035 {
00036
00037     enum data_t {
00038       nucleotide,
00039       codon,
00040       protein
00041     };
00042
00043     enum local_t {
00044       trim,
00045       global,
00046       local
00047     };
00048
00049     enum space_t {
00050         quadratic,
00051         linear
00052     };
00053
00054     enum out_format_t {
00055         refmap,
00056         refalign,
00057         pairwise
00058     };
00059
00060     enum rc_t {
00061         none,
00062         silent,
00063         annotated
00064     };
00065
00066    class args_t {
00067     public:
00068
00069         FILE            * output,
00070                         * reference,
00071                         * input;
00072
00073         ConfigParser    * scores;
00074
00075         data_t          data_type;
00076         local_t         local_option;
00077         space_t         space_type;
00078         out_format_t    out_format;
00079         rc_t            reverse_complement;
00080         ConfigParser    * genetic_code;
00081
00082         bool            quiet;
00083         bool            affine;
00084         bool            include_reference;
00085
00086        StringBuffer*   memory_ref;
00087
00088
00089         args_t( int, const char ** );
00090         ~args_t();
00091
00092     private:
00093         void parse_input        ( const char * );
00094         void parse_reference    ( const char * );
00095         void parse_output       ( const char * );
00096         void parse_scores       ( const char * );
00097         void parse_quiet        ( void );
00098         void parse_affine       ( void );
00099         void parse_include_ref  ( void );
00100         void parse_rc           ( const char * );
00101         void parse_space_t      ( const char * );
00102         void parse_data_t       ( const char * );
00103         void parse_local_t      ( const char * );
00104         void parse_out_format_t ( const char * );
00105         void parse_genetic_code ( const char * );
00106
00107         std::string code_name;
00108
00109     };
00110
00111     void ERROR_NO_USAGE ( const char * msg, ... );
00112
00113     // defined in argparse.cpp
00114     std::ifstream check_file_path_stream (const char* path, const char * subpath);
00115 }
00116
00117 #endif // ARGPARSE_H
```

5.3 configparser.hpp

```
00001 // Copyright (c) 2018 Daniel Zilles
00002 //
00003 // Permission is hereby granted, free of charge, to any person obtaining a copy
00004 // of this software and associated documentation files (the "Software"), to deal
00005 // in the Software without restriction, including without limitation the rights
00006 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00007 // copies of the Software, and to permit persons to whom the Software is
00008 // furnished to do so, subject to the following conditions:
00009 //
00010 // The above copyright notice and this permission notice shall be included in all
00011 // copies or substantial portions of the Software.
00012 //
00013 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00014 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00015 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00016 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00017 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00018 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00019 // SOFTWARE.
00020
00021 #ifndef CONFIGPARSER_HPP
00022 #define CONFIGPARSER_HPP
00023
00024 #include <string>
00025 #include <vector>
00026 #include <map>
00027 #include <iostream>
00028 #include <sstream>
00029
00030 typedef std::map<std::string, std::vector<std::string» configList;
00031
00032 class ConfigParser {
00033
00034   public:
00035   ConfigParser(std::ifstream& configFile);
00036
00037     bool hasConfig(std::string section, std::string name) const {
00038         return mConfigurations.find(section + " - " + name) != mConfigurations.end();
00039     }
00040     template<typename T>
00041     T aConfig(std::string section, std::string name, size_t pos = 0);
00042     template<typename T>
00043     std::vector<T> aConfigVec(std::string section, std::string name);
00044
00045
00046   private:
00047     static void       handleMissingKey (std::string);
00048     configList mConfigurations;
00049
00050 };
00051
00052 template <>
00053 bool ConfigParser::aConfig<bool>(std::string section, std::string name, size_t pos);
00054
00055 template <typename T>
00056 T ConfigParser::aConfig(std::string section, std::string configName, size_t pos) {
00057
00058   T tmp;
00059
00060   const auto& mConfigRef = mConfigurations;
00061   auto search = mConfigRef.find(section + " - " + configName);
00062
00063   if (search == mConfigRef.end()) {
00064       handleMissingKey (std::string("Could not find required configuration section ") + section +
      std::string(" key ") + configName);
00065   }
00066
00067   std::istringstream iss(search->second[0]);
00068
00069   if (search->second[0].find( "0x" ) != std::string::npos)
00070     iss » std::hex » tmp;
00071   else
00072     iss » std::dec » tmp;
00073
00074   return tmp;
00075 }
00076
00077 template <>
00078 std::vector<bool> ConfigParser::aConfigVec<bool>(std::string section, std::string name);
00079
00080 template <typename T>
00081 std::vector<T> ConfigParser::aConfigVec(std::string section, std::string configName) {
00082
00083
```

```
00084   const auto& mConfigRef = mConfigurations;
00085   auto search = mConfigRef.find(section + " - " + configName);
00086
00087   if (search == mConfigRef.end()) {
00088     handleMissingKey (std::string("Could not find required configuration section ") + section +
      std::string(" key ") + configName);
00089   }
00090
00091
00092   std::vector<T>  tmp(search->second.size());
00093   for (unsigned i = 0; i < search->second.size(); i++) {
00094
00095     std::istringstream iss(search->second[i]);
00096
00097     if (search->second[i].find( "0x" ) != std::string::npos)
00098       iss » std::hex » tmp[i];
00099     else
00100       iss » std::dec » tmp[i];
00101   }
00102   return tmp;
00103 }
00104 #endif
```

5.4 scoring.hpp

```
00001 #ifndef SCORING_H
00002 #define SCORING_H
00003
00004 #include <iostream>
00005 #include "alignment.h"
00006 #include "argparse.hpp"
00007 #include "tn93_shared.h"
00008
00009 using namespace std;
00010 using namespace argparse;
00011
00012
00013 class CawalignSimpleScores {
00014     public:
00015         CawalignSimpleScores  (
00016                                 const char * _alphabet,
00017                                 const cawlign_fp * _scoring_matrix,
00018                                 const cawlign_fp _open_gap_reference,
00019                                 const cawlign_fp _open_gap_query,
00020                                 const cawlign_fp _extend_gap_reference,
00021                                 const cawlign_fp _extend_gap_query
00022                               );
00023
00024         CawalignSimpleScores  (ConfigParser*);
00025         CawalignSimpleScores  (void) {D=0;};
00026         virtual ~CawalignSimpleScores (void) {};
00027
00028         StringBuffer           alphabet;
00029         /*
00030             ordered characters that are included in the scoring matrix
00031         */
00032         unsigned int          D;
00033         // the number of characters in the string
00034
00035         long                  char_map [255];
00036         /*
00037             for each ASCII character, this will map the character to the corresping entry the scoring
      matrix
00038             all characters NOT in `alphabet' get mapped to index D (the 'not defined' character)
00039         */
00040
00041         VectorFP         scoring_matrix;
00042         /*
00043                 A (D+1)x(D+1) scoring matrix where element (i,j) gives the score of matching (or
      mis-matching)
00044                 the D-th row/column is for matchign a character NOT in the alphabet
00045                 While generally symmetric, an asymmetric matrix can also be meaningful if there is
      some reason to have
00046                 substitutions in reference/query weigted differently
00047         */
00048
00049         cawlign_fp          open_gap_reference,
00050                             open_gap_query,
00051                             extend_gap_query,
00052                             extend_gap_reference;
00053
00054         char                gap_char;
00055         /* gap open and extend character*/
```

```
00056
00057         void                _init_alphabet (long not_found = -1);
00058
00059 };
00060
00061 class CawalignCodonScores : public CawalignSimpleScores {
00062     public:
00063
00064         CawalignCodonScores  (ConfigParser*, ConfigParser*);
00065         virtual ~CawalignCodonScores (void) {};
00066
00067         // compute how many nucleotides are different between the two codons encoded as 0-63 integers
00068         static int nucleotide_diff (long, long);
00069
00070         Vector                translation_table,
00071         // codon (0-63 index) to single letter amino-acid code translation table
00072                               resolutions;
00073         // an linear array of resolutions for ambiguous nucleotides
00074         // if char_map[i] < -1, then indexing into this array using (-char_map[i]-2)*4 index will
00075         // return a four integer map, with 0 indicating that the corresponding character is not in the
      resolution, and 1 - that it is
00076
00077         // partial score tables
00078         VectorFP          s3x1,
00079                               s3x2,
00080                               s3x4,
00081                               s3x5;
00082
00083         // the cost of introducing frameshits
00084         cawlign_fp                frameshift_cost,
00085         // the penalty for synonymous substitutions, per nucleotide change
00086                               synonymous_penalty;
00087
00088         // ordered amino-acid scoring tables
00089         StringBuffer          amino_acids;
00090
00091         int                   stop_codon_index;
00092         int                   mismatch_index;
00093
00094
00095
00096 };
00097
00098
00099 extern const char   kNucleotideAlphabet[];
00100 extern const cawlign_fp kNucScoring[];
00101
00102 #endif
```

5.5 stringBuffer.h

```
00001 #ifndef __STRINGBUFFER__
00002 #define __STRINGBUFFER__
00003
00004 #include "alignment.h"
00005
00006 //____________________________________________________________________________________
00007
00012 class StringBuffer {
00013
00014   char *sData;
00015   unsigned long sLength, saLength;
00016
00017 public:
00022   StringBuffer(void);
00027   ~StringBuffer(void);
00028
00034   char *getString(void) const { return sData; }
00040   void appendChar(const char);
00047   void appendBuffer(const char *, const long = -1);
00052   void resetString(void);
00058   void swap(StringBuffer &);
00064   unsigned long length(void) const { return sLength; }
00070   void reset_length(unsigned long newL) {
00071     if (newL < sLength) {
00072       sLength = newL;
00073     }
00074   }
00075
00083   char setChar(const long i, const char c) {
00084     char oc = sData[i];
00085     sData[i] = c;
00086     return oc;
```

```
00087   }
00088
00095   char getChar(const long i) const { return sData[i]; }
00100   void flip (void);
00105   void detach (void) { sData = nullptr;}
00106
00107   static long sbDefaultLength, sbDefaultBoost;
00108 };
00109
00110 //____________________________________________________________________________________
00111
00116 class VectorFP {
00117
00118   cawlign_fp *vData;
00119
00120   unsigned long vLength, vaLength;
00121
00122 public:
00127   VectorFP(void);
00132   ~VectorFP(void);
00133
00139   void appendValue(const cawlign_fp);
00146   void appendValues(const cawlign_fp*, long);
00153   void storeValue(const cawlign_fp, const unsigned long);
00160   cawlign_fp value(const long idx) { return vData[idx]; }
00166   unsigned long length(void) const { return vLength; }
00172   const cawlign_fp * values (void) {return vData;}
00178   cawlign_fp * rvalues (void) {return vData;}
00179
00180   static long vDefaultLength, vDefaultBoost;
00181 };
00182
00183 //____________________________________________________________________________________
00184
00189 class Vector {
00190
00191   long *vData;
00192
00193   unsigned long vLength, vaLength;
00194
00195 public:
00200   Vector(void);
00205   ~Vector(void);
00206
00212   void appendValue(const long);
00218   void appendVector(const Vector &);
00225   long extractMin(VectorFP &);
00230   void resetVector(void);
00236   void remove(const unsigned long);
00243   void storeValue(const long, const unsigned long);
00250   void storeVector(const Vector &, const unsigned long);
00255   void sort(void);
00261   void swap(Vector &);
00268   long value(const long idx) const;
00274   unsigned long length(void) const { return vLength; }
00275
00276   const long * rvalues (void) const {return vData;}
00277
00278   static long vDefaultLength, vDefaultBoost;
00279 };
00280
00281 #endif
```

5.6 tn93_shared.h

```
00001 #ifndef     __TN93SHARED__
00002 #define     __TN93SHARED__
00003
00004 #include <iostream>
00005 #include <cstdlib>
00006 #include <cstdio>
00007 #include <iomanip>
00008 #include <math.h>
00009 #include <string.h>
00010 #include <unistd.h>
00011 #include <climits>
00012 #include <cstdint>
00013 #include "stringBuffer.h"
00014
00015 using namespace std;
00016
00017 #define MIN(a,b) (a) < (b) ? (a) : (b)
00018 #define MAX(a,b) (a) > (b) ? (a) : (b)
```

```
00019
00030 int readFASTA (FILE* F, char& automatonState, StringBuffer &names, StringBuffer& sequences, long&
      firstSequenceLength);
00036 void initAlphabets(bool doAminoAcid = false);
00045 int    reverseComplement (StringBuffer& sequence, unsigned long from, unsigned long to);
00046
00047 extern int8_t validFlags[];
00048
00049 #endif
```

Index

_init_alphabet
    CawalignSimpleScores, 11
~StringBuffer
    StringBuffer, 15
~Vector
    Vector, 20
~VectorFP
    VectorFP, 26
~args_t
    argparse::args_t, 8

aConfig
    ConfigParser, 13
aConfigVec
    ConfigParser, 13
appendBuffer
    StringBuffer, 16
appendChar
    StringBuffer, 16
appendValue
    Vector, 21
    VectorFP, 26
appendValues
    VectorFP, 27
appendVector
    Vector, 21
argparse::args_t, 7
    ~args_t, 8
    args_t, 7
args_t
    argparse::args_t, 7

CawalignCodonScores, 8
    CawalignCodonScores, 9
    nucleotide_diff, 10
CawalignSimpleScores, 10
    _init_alphabet, 11
    CawalignSimpleScores, 11
ConfigParser, 12
    aConfig, 13
    aConfigVec, 13
    ConfigParser, 12

extractMin
    Vector, 21

flip
    StringBuffer, 17

getChar
    StringBuffer, 17
getString
    StringBuffer, 17

KmerHit, 14

length
    StringBuffer, 17
    Vector, 22
    VectorFP, 27

nucleotide_diff
    CawalignCodonScores, 10

RefKmerEntry, 14
remove
    Vector, 22
reset_length
    StringBuffer, 17
resetString
    StringBuffer, 18
resetVector
    Vector, 23
rvalues
    VectorFP, 27

setChar
    StringBuffer, 18
sort
    Vector, 23
src/alignment.h, 29
src/argparse.hpp, 30
src/configparser.hpp, 32
src/scoring.hpp, 33
src/stringBuffer.h, 34
src/tn93_shared.h, 35
storeValue
    Vector, 23
    VectorFP, 27
storeVector
    Vector, 23
StringBuffer, 14
    ~StringBuffer, 15
    appendBuffer, 16
    appendChar, 16
    flip, 17
    getChar, 17
    getString, 17
    length, 17
    reset_length, 17
    resetString, 18
    setChar, 18

StringBuffer, 15
swap, 18
swap
StringBuffer, 18
Vector, 24

value
Vector, 24
VectorFP, 28
values
VectorFP, 28
Vector, 19
∼Vector, 20
appendValue, 21
appendVector, 21
extractMin, 21
length, 22
remove, 22
resetVector, 23
sort, 23
storeValue, 23
storeVector, 23
swap, 24
value, 24
Vector, 20
VectorFP, 25
∼VectorFP, 26
appendValue, 26
appendValues, 27
length, 27
rvalues, 27
storeValue, 27
value, 28
values, 28
VectorFP, 26